RustTermView (RTV)

SPECYFIKACJA TECHNICZNA I PLAN IMPLEMENTACJI PROJEKTU

Język implementacji	Rust (wymóg bezwzględny)	**Status dokumentu**	Zatwierdzony / Finalny
Typ projektu	Aplikacja CLI / Grafika terminalowa	**Target OS**	Linux (Ubuntu)

1. Cel i Założenia Projektu

Celem projektu jest stworzenie zaawansowanego narzędzia wiersza poleceń (CLI) o nazwie **RustTermView (RTV)**, napisanego w języku Rust. Aplikacja służy do renderowania obrazów statycznych (w formatach PNG, JPEG, BMP) oraz animacji (GIF) bezpośrednio wewnątrz okna terminala systemowego.

Projekt kładzie szczególny nacisk na poprawne zarządzanie zasobami, wydajność operacji wejścia/wyjścia (I/O) oraz asynchroniczne przetwarzanie zdarzeń systemowych, co pozwoli w pełni wykorzystać mechanizmy bezpieczeństwa i współbieżności oferowane przez język Rust.

2. Architektura i Struktura Modułowa

Kod źródłowy zostanie podzielony na logiczne, niezależne moduły. Zapewni to wysoką czytelność, łatwość testowania jednostkowego oraz umożliwi elastyczny rozwój bazy kodu.

```
src/
├── main.rs          # Punkt wejścia, inicjalizacja i główna pętla programu
├── args.rs          # Definicje struktur i flag wiersza poleceń (Clap)
├── terminal.rs      # Obsługa rozmiaru okna, trybu Raw Mode oraz sygnałów systemowych
├── loader.rs        # Odczyt plików, dekodowanie formatów graficznych i klatek GIF
├── transform.rs     # Algorytmy skalowania, konwersji kolorów i filtrów cyfrowych
└── render.rs        # Formatowanie bufora wyjściowego i generowanie sekwencji ANSI/ASCII
```

3. Szczegółowy Opis Funkcjonalności

3.1. Dynamiczne dopasowanie do geometrii terminala

Program automatycznie pobiera wymiary okna konsoli w momencie uruchomienia (oraz w trakcie odtwarzania animacji), wykorzystując bibliotekę `crossterm`. Obraz jest skalowany w taki sposób, aby maksymalnie wypełnić przestrzeń, nie powodując przy tym zawijania linii tekstu.

> **Uwzględnienie proporcji znaków (Aspect Ratio):**
> Standardowy znak w terminalu (czcionka o stałej szerokości) ma zazwyczaj proporcje pionowe do poziomych wynoszące około 1:2 (jest dwukrotnie wyższy niż szerszy). Aby uniknąć deformacji (pionowego rozciągnięcia) obrazu, algorytm skalujący musi skorygować docelową wysokość, mnożąc ją przez współczynnik $k = 0.5$ przed wykonaniem operacji resizingu.

3.2. Zaawansowane Tryby Wyświetlania i Filtry

- **Tryb Truecolor 24-bit (Standard):** Renderowanie pikseli przy użyciu spacji ze zdefiniowanym kolorem tła za pomocą sekwencji ucieczki ANSI: `\x1b[48;2;R;G;Bm \x1b[0m`.
- **Tryb Wysokiej Rozdzielczości (Pół-bloki / Half-Blocks):** Podwojenie rozdzielczości pionowej poprzez wykorzystanie znaku Unicode górnej połowy bloku ▀ (`\u{2580}`). Kolor czcionki (Foreground) odpowiada górnemu pikselowi obrazu, a kolor tła (Background) odpowiada pikselowi dolnemu. Sekwencja przyjmuje postać: `\x1b[38;2;R1;G1;B1;48;2;R2;G2;B2m▀\x1b[0m`.
- **Tryb Skali Szarości (Grayscale):** Konwersja kolorów RGB do monochromatycznych na podstawie luminancji wyliczanej wzorem:

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

  Wartość Y mapowana jest na dedykowaną, 24-stopniową paletę odcieni szarości standardu ANSI (indeksy od 232 do 255).
- **Tryb ASCII Art:** Mapowanie obliczonej jasności piksela na znaki tekstowe o zróżnicowanej gęstości optycznej (np. od spacji dla bieli do znaku `@` dla pełnej czerni).

4. Obsługa Animacji GIF oraz Pokazów Slajdów

Program umożliwia sekwencyjne wyświetlanie obrazów z katalogu (pokaz slajdów) oraz natywne odtwarzanie animacji w formacie GIF poprzez dekodowanie pojedynczych klatek za pomocą `image::codecs::gif::GifDecoder`.

Kluczowym wyzwaniem inżynierskim jest zapobieganie blokowaniu programu. Klasyczne usypianie wątku (`std::thread::sleep`) uniemożliwia natychmiastowe przechwycenie sygnałów użytkownika. Projekt implementuje **asynchroniczną pętlę zdarzeń (Non-blocking Event Loop)** opartą na nieblokującym odpytywaniu o zdarzenia (`crossterm::event::poll`).

Dzięki temu rozwiązaniu naciśnięcie klawisza `q` lub kombinacji `Ctrl+C` przerywa odtwarzanie w sposób natychmiastowy, pozwalając na bezpieczne przywrócenie stanu terminala (wyłączenie trybu surowego *Raw Mode* oraz ponowne wyświetlenie kursora myszy).

5. Optymalizacja Wydajnościowa

W celu wyeliminowania efektu "klatkowania" oraz widocznego, linijka po linijce, rysowania obrazu, aplikacja stosuje pełne buforowanie wyjścia. Zamiast wielokrotnego wywoływania makra `print!` dla pojedynczych pikseli, cała klatka obrazu jest transformowana do jednego, ciągłego obiektu typu `String` w pamięci RAM lub strumieniowana przy użyciu struktury `std::io::BufWriter`. Wypchnięcie danych do standardowego wyjścia (stdout) następuje raz na klatkę, co pozwala na osiągnięcie płynności renderowania na poziomie przekraczającym 60 FPS.

6. Plan Implementacji (Harmonogram 7 Tygodni)

Prace nad projektem zostały ustrukturyzowane w postaci jednolitego planu obejmującego cztery główne fazy, prowadzące od wczesnego prototypu aż do w pełni funkcjonalnego narzędzia.

1. **Faza 1: Podstawy i Proof of Concept (Tydzień 1 - 2)**
   Konfiguracja środowiska, inicjalizacja repozytorium Cargo. Implementacja mechanizmu wczytywania plików przez bibliotekę `image` i ekstrakcja tablic pikseli RGB. Opracowanie podstawowego modułu sterowania terminalem za pomocą kodów ANSI (wyświetlanie podstawowych struktur testowych).
2. **Faza 2: Algorytmika i Interfejs CLI (Tydzień 3 - 4)**
   Stworzenie funkcji skalującej (algorytm Nearest Neighbor) uwzględniającej korekcję aspect ratio. Integracja z biblioteką `crossterm` w celu automatycznego pobierania wymiarów okna. Przygotowanie struktur parsowania linii poleceń (biblioteka `clap`).
3. **Faza 3: Integracja Systemowa - MVP (Tydzień 5)**
   Połączenie modułów ładowania, transformacji i renderowania w jeden spójny potok danych. Implementacja wyświetlania pojedynczego zdjęcia podanego jako argument CLI. Debugowanie błędów konwersji i optymalizacja bufora I/O.

4. **Faza 4: Tryby Zaawansowane i Szlify (Tydzień 6 - 7)**
   Implementacja nieblokującej pętli odtwarzania GIF-ów oraz obsługi pokazów slajdów. Dodanie filtrów graficznych: pół-bloki (Half-Blocks), skala szarości, ASCII Art. Zapewnienie pełnej obsługi błędów systemowych (brak pliku, niepoprawny format) oraz finalna kompilacja optymalizacyjna (`--release`).

7. Konfiguracja Środowiska Zależności (`Cargo.toml`)

Projekt wykorzystuje wyłącznie stabilne, wysoko zoptymalizowane biblioteki ze społeczności Rusta:

```
[package]
name = "rust_term_view"
version = "0.1.0"
edition = "2021"

[dependencies]
image = "0.25"       # Dekodowanie formatów graficznych oraz klatek animacji GIF
crossterm = "0.27"   # Zarządzanie terminalem, tryb Raw Mode, asynchroniczne eventy
clap = { version = "4.4", features = ["derive"] } # Zaawansowany parser CLI
```